

 Práctica Evaluativa –Parcial I

Asignatura: Programación I

Tema: Estructuras Repetitivas con Listas

Caso 11 – Taller – Órdenes de reparación y tiempos estimados

Enunciado / Descripción

Un taller mecánico desea un sistema para gestionar las órdenes de reparación y los tiempos estimados para cada una. Se pide construir un menú que utilice listas paralelas: una lista `ordenes[]` para almacenar los códigos de las órdenes (ej., "ORD-001", "ORD-002") y otra lista `horas[]` para almacenar el tiempo estimado en horas para cada orden. Ambas listas deben compartir el índice, de manera que el tiempo estimado en `horas[i]` corresponda a la orden en `ordenes[i]`. El programa debe usar un bucle `while` para presentar un menú al usuario, permitiéndole realizar diferentes operaciones hasta que elija la opción "Salir".

Ejemplo:

- `ordenes[] = ["ORD-001", "ORD-002", "ORD-003"]`
- `horas[] = [2.5, 0, 4.0]`

En este ejemplo:

- La orden "ORD-001" tiene un tiempo estimado de 2.5 horas.
- La orden "ORD-002" tiene un tiempo estimado de 0 horas (pendiente de diagnóstico).
- La orden "ORD-003" tiene un tiempo estimado de 4.0 horas.

Desafío: Implementar el sistema de gestión de órdenes y tiempos estimados utilizando listas paralelas y un menú interactivo.

Menú:

1. Ingresar órdenes (códigos): (Registrar las órdenes de reparación en el sistema)
 - Permite al usuario ingresar los códigos de las órdenes de reparación.
 - Ejemplo: El usuario ingresa "ORD-004", "ORD-005".
2. Ingresar horas estimadas por orden: (Definir el tiempo estimado inicial para cada orden)
 - Permite al usuario ingresar el tiempo estimado en horas para cada orden, utilizando valores numéricos (puede ser decimal). Debe corresponder al orden de las órdenes ingresadas previamente.

- Ejemplo: Si las órdenes son "ORD-004" y "ORD-005", el usuario podría ingresar "3.0" para ORD-004 y "1.5" para ORD-005.
- 3. Mostrar agenda del taller: (Mostrar todas las órdenes y sus tiempos estimados)
 - Muestra una lista de todas las órdenes y su tiempo estimado correspondiente.
 - Ejemplo de salida:
 - "Orden: ORD-001, Tiempo estimado: 2.5 horas"
 - "Orden: ORD-002, Tiempo estimado: 0 horas"
 - "Orden: ORD-003, Tiempo estimado: 4.0 horas"
 - "Orden: ORD-004, Tiempo estimado: 3.0 horas"
 - "Orden: ORD-005, Tiempo estimado: 1.5 horas"
- 4. Consultar horas por orden: (Verificar el tiempo estimado para una orden específica)
 - Permite al usuario ingresar el código de una orden y ver su tiempo estimado actual.
 - Ejemplo: El usuario ingresa "ORD-002" y el programa muestra "Orden: ORD-002, Tiempo estimado: 0 horas".
- 5. Listar órdenes con 0 horas (pendiente de diagnóstico): (Mostrar las órdenes que requieren diagnóstico)
 - Muestra una lista reducida que solo incluye las órdenes con un tiempo estimado de 0 horas.
- 6. Agregar orden: (Añadir una nueva orden al sistema)
 - Permite al usuario agregar una nueva orden al sistema, asignándole un tiempo estimado inicial.
- 7. Actualizar horas: (Modificar el tiempo estimado de una orden)
 - Permite al usuario cambiar el tiempo estimado de una orden. Debe solicitar el código de la orden y el nuevo tiempo estimado deseado.
- 8. Salir: (Terminar el programa)
 - Termina la ejecución del programa.

Notas adicionales:

- **Considerar validaciones de entrada (ej., asegurarse de que las horas ingresadas sean números positivos).**

- Al agregar una orden, verificar que el código no exista ya.
- Al actualizar las horas, verificar que la orden exista.



Entregables

El estudiante deberá **subir el archivo del programa en lenguaje Python** a la plataforma institucional. NO SUBIR UN REPOSITORIO DE GITHUB. **SOLO SUBIR EL ARCHIVO.PY**

El código debe cumplir con:

- Todas las funcionalidades solicitadas reflejadas en el menú.
- Buena ejecución sin errores.
- Nomenclatura clara en el nombre de las variables.
- Legibilidad general y buenas prácticas de codificación.



Rúbrica de Evaluación

Rúbrica de Evaluación Detallada para "Órdenes de Reparación y Tiempos Estimados"

Código	Criterio	Peso	Descripción Detallada
C1	Correctitud Funcional	50%	Este criterio evalúa la correcta implementación de la funcionalidad central del programa. Esto incluye: • Agregar orden: La capacidad de añadir nuevas órdenes de reparación al sistema, validando que el código de la orden sea único. • Ver sin estimación: La capacidad de listar correctamente las órdenes que actualmente no tienen un tiempo estimado asignado (horas = 0). • Actualizar horas: La capacidad de modificar el tiempo estimado de una orden existente, validando que la orden exista y que el nuevo tiempo estimado sea válido (ej., no negativo). • Ver agenda: La capacidad de mostrar una lista completa de todas las órdenes y sus correspondientes tiempos estimados. • Consulta específica: La capacidad de buscar y mostrar el tiempo estimado para una orden específica a partir de su código. • Manejo de casos extremos: La capacidad del programa de manejar situaciones inesperadas o valores límite de forma correcta, sin errores ni comportamientos anómalos. Por ejemplo, ¿qué ocurre si se intenta

Código	Criterio	Peso	Descripción Detallada
			agregar una orden con un código que ya existe? ¿O si se intenta actualizar las horas de una orden que no existe?
C2	Cumplimiento de Restricciones	20%	Este criterio evalúa el cumplimiento de las restricciones específicas del problema, diseñadas para guiar la implementación hacia una solución correcta y eficiente. Esto incluye: • Listas paralelas: El uso correcto y obligatorio de listas paralelas para mantener la correspondencia entre códigos de órdenes y tiempos estimados. • Conservar órdenes aunque horas = 0: El sistema debe mantener las órdenes aunque su tiempo estimado sea cero, en lugar de eliminarlas. Esto permite, por ejemplo, diagnosticar la orden más adelante y asignarle un tiempo estimado. • Sin estructuras prohibidas: El uso de estructuras de datos no permitidas (ej., diccionarios, clases) resultará en una penalización. El objetivo es evaluar la capacidad de resolver el problema con las herramientas específicamente indicadas (listas paralelas). • Sincronía entre listas: El programa debe asegurar que la sincronización entre las listas ordenes[] y horas[] se mantenga en todo momento. El elemento en la posición i de una lista debe corresponder al elemento en la posición i de la otra lista.
C3	Interacción y Validación	10%	Este criterio evalúa la calidad de la interacción del programa con el usuario y la robustez de las validaciones de entrada. Esto incluye: • Código no vacío: El programa debe validar que el código de una orden ingresada por el usuario no esté vacío. • Duplicado al agregar: El programa debe verificar que el código de la orden que se intenta agregar no exista ya en la lista. • Horas iniciales ≥ 0: El programa debe validar que el tiempo estimado inicial ingresado por el usuario sea un número no negativo. • Actualización ≥ 0: Al actualizar las horas, el programa debe validar que el nuevo tiempo estimado ingresado sea un número no negativo. • Orden inexistente: Al consultar o actualizar una orden, el programa debe verificar que la orden exista. • Menú persistente: El menú debe permanecer visible y funcional hasta que el usuario explícitamente elija la

Código	Criterio	Peso	Descripción Detallada
			opción "Salir". Opciones inválidas: El programa debe manejar las entradas inválidas del usuario (ej., ingresar una letra en lugar de un número en el menú) de forma elegante, sin crashearse, y mostrando un mensaje de error apropiado.
C4	Estructura y Legibilidad	10%	Este criterio evalúa la claridad, organización y calidad general del código. Esto incluye: - Variables descriptivas: El uso de nombres de variables que sean claros, significativos y representativos de los datos que almacenan. - Estructura clara: El código debe estar bien organizado y estructurado, con una lógica clara y fácil de seguir. Usar indentación consistente y comentarios cuando sea necesario para aclarar la lógica. - Mensajes coherentes: Los mensajes que el programa muestra al usuario deben ser claros, concisos y fáciles de entender. - Evitar repetición: El código debe evitar la duplicación innecesaria de código. Utilizar funciones para reutilizar la lógica común.
C5	Casos de Prueba / Datos de Ejemplo	5%	Este criterio evalúa la exhaustividad y relevancia de los casos de prueba utilizados para demostrar la funcionalidad del programa. Esto incluye: - Horas = 0: Incluir un caso de prueba que demuestre el funcionamiento con una orden que no tiene tiempo estimado asignado. - Actualizar a valor negativo (no permitido): Intentar actualizar el tiempo estimado a un valor negativo. - Consulta inexistente: Intentar consultar el tiempo estimado de una orden que no existe. - Etc.: Incluir otros casos que permitan probar diferentes escenarios y bordes del programa.
C6	Gestión de Casos Borde	5%	Este criterio evalúa la capacidad del programa para manejar correctamente situaciones límite o casos especiales que pueden surgir durante la ejecución. Esto incluye: - Orden inexistente: Intentar consultar o actualizar una orden que no existe. - Horas negativas: Intentar ingresar un tiempo estimado negativo. - Duplicados: Intentar agregar una orden con un código que ya existe. - Código vacío: Intentar agregar una orden con un código vacío.

Descriptores globales por nivel

Excelente (90–100): Cumple todos los criterios sin fallas; maneja casos borde exhaustivamente; interacción robusta; código claro y consistente.

Muy Bueno (80–89): Cumple la mayoría; pequeños desajustes no críticos; validación adecuada; estilo mayormente claro.

Aprobado (60–79): Cumple los básicos; algunos fallos funcionales menores o validación incompleta; legibilidad aceptable.

Insuficiente (<60): Incumple criterios clave; errores que impiden el uso; omite restricciones solicitadas.

Penalizaciones y observaciones

- –30% si utiliza estructuras prohibidas (diccionarios, clases, etc.).
- –10% si no conserva ítems con cantidad 0 (elimina de una lista y no de la otra).
- –10% por ausencia total de validación de entradas.
- –5% por mensajes ininteligibles o menú que no persiste.

Notas

- Asignar puntajes parciales por criterio y calcular la nota final ponderada.
- Compartir la rúbrica con estudiantes antes de la evaluación.
- Mantener consistencia entre enunciados, resultados de aprendizaje y criterios de evaluación.



Entregables

El estudiante deberá **subir el archivo del programa en lenguaje Python** a la plataforma institucional. NO SUBIR UN REPOSITORIO DE GITHUB. **SOLO SUBIR EL ARCHIVO.PY**

El código debe cumplir con:

- Todas las funcionalidades solicitadas reflejadas en el menú.
- Buena ejecución sin errores.
- Nomenclatura clara en el nombre de las variables.
- Legibilidad general y buenas prácticas de codificación.



Rúbrica de Evaluación

Rúbrica adaptada

Código	Criterio	Peso
C1 Correctitud funcional	50%	Agregar orden; ver sin estimación; actualizar horas; ver agenda; consulta específica; sin errores en borde.
C2 Cumplimiento de restricciones	20%	Listas paralelas; conservar órdenes aunque horas = 0; sin estructuras prohibidas; sincronía entre ordenes[] y horas[].
C3 Interacción y validación	10%	Código no vacío; duplicado al agregar; horas iniciales ≥ 0 ; actualización ≥ 0 ; orden inexistente; menú persistente; opciones inválidas.
C4 Estructura y legibilidad	10%	Variables descriptivas; estructura clara; mensajes coherentes; evitar repetición.
C5 Casos de prueba / Datos de ejemplo	5%	Horas = 0; actualizar a valor negativo (no permitido); consulta inexistente; etc.
C6 Gestión de casos borde	5%	Orden inexistente; horas negativas; duplicados; código vacío.

Descriptorios globales por nivel

Excelente (90–100): Cumple todos los criterios sin fallas; maneja casos borde exhaustivamente; interacción robusta; código claro y consistente.

Muy Bueno (80–89): Cumple la mayoría; pequeños desajustes no críticos; validación adecuada; estilo mayormente claro.

Aprobado (60–79): Cumple los básicos; algunos fallos funcionales menores o validación incompleta; legibilidad aceptable.

Insuficiente (<60): Incumple criterios clave; errores que impiden el uso; omite restricciones solicitadas.

Penalizaciones y observaciones

- –30% si utiliza estructuras prohibidas (diccionarios, clases, etc.).
- –10% si no conserva ítems con cantidad 0 (elimina de una lista y no de la otra).
- –10% por ausencia total de validación de entradas.
- –5% por mensajes ininteligibles o menú que no persiste.

Notas

- Asignar puntajes parciales por criterio y calcular la nota final ponderada.
- Compartir la rúbrica con estudiantes antes de la evaluación.
- Mantener consistencia entre enunciados, resultados de aprendizaje y criterios de evaluación.

